



MoreMarkets Vault

Security Review



MORE

Disclaimer

Security Review

MoreMarkets Vault



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

MoreMarkets Vault

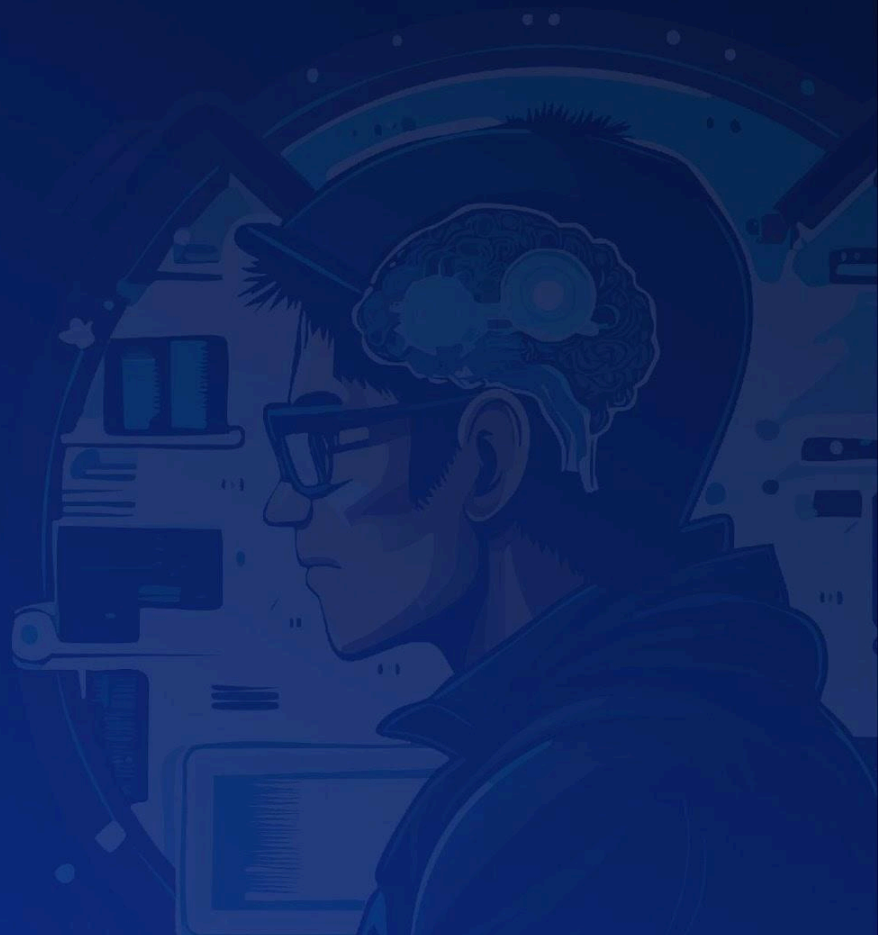


Table of Contents

Disclaimer	3
Summary	7
Scope	9
Methodology	9
Project Dashboard	13
Risk Section	16
Findings	18
3S-Vault-M01	18
3S-Vault-L01	20
3S-Vault-L02	21
3S-Vault-L03	22
3S-Vault-L04	23
3S-Vault-N01	24

Summary Security Review

MoreMarkets Vault



Summary

Three Sigma audited MoreMarkets in a 0.8 person week engagement. The audit was conducted from 25/03/2026 to 26/03/2026.

Protocol Description

MORE Vaults is an open-source cross-chain vault standard that combines the composability of the ERC-2535 Diamond Standard with a rigorously audited, invariant and generic core. By separating the asset-critical logic from the ever-evolving strategy layer, MORE drives down implementation and security costs for strategists, integrated protocols, and depositors alike.

Scope Security Review

MoreMarkets Vault

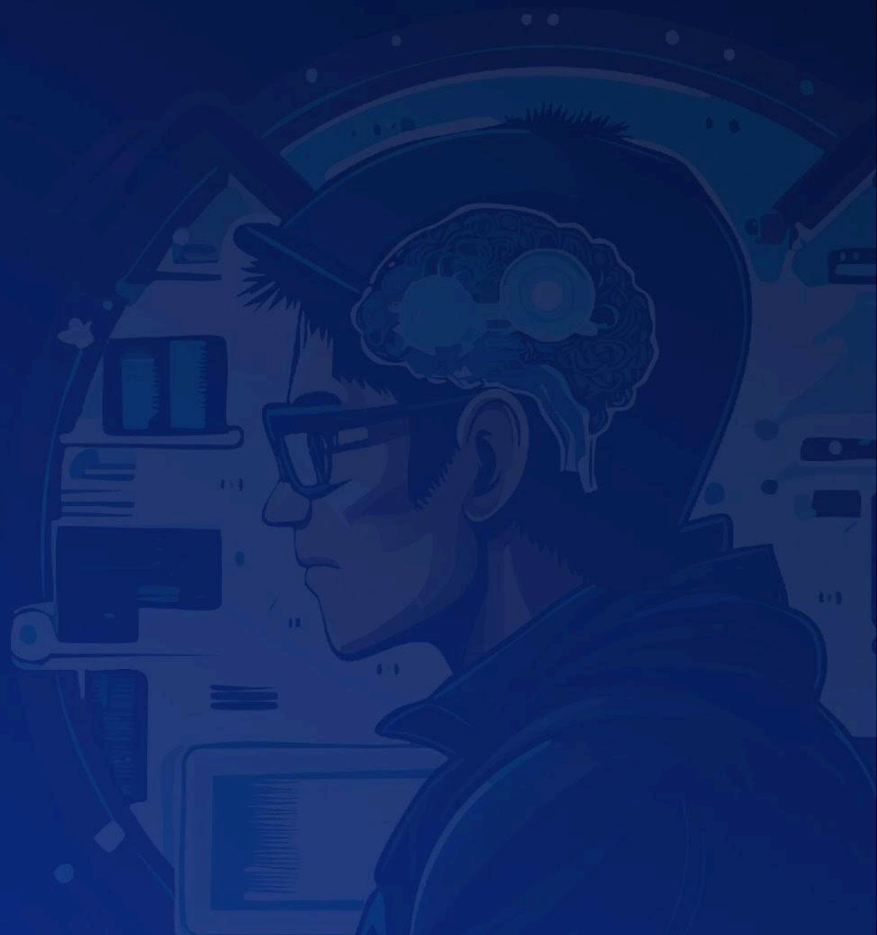


Scope

Filepath	nSL0C
src/periphery/MoreVaultMigrator.sol	72
Total	72

Methodology Security Review

MoreMarkets Vault



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard Security Review

MoreMarkets Vault



Project Dashboard

Application Summary

Name	MoreMarkets
Repository	https://github.com/MORE-Vaults/MORE-Vaults-Core
Commit	f0ea5cb
Language	Solidity
Platform	Flow

Engagement Summary

Timeline	25/03/2026 to 26/03/2026
Nº of Auditors	2
Review Time	0.8 person weeks

Vulnerability Summary

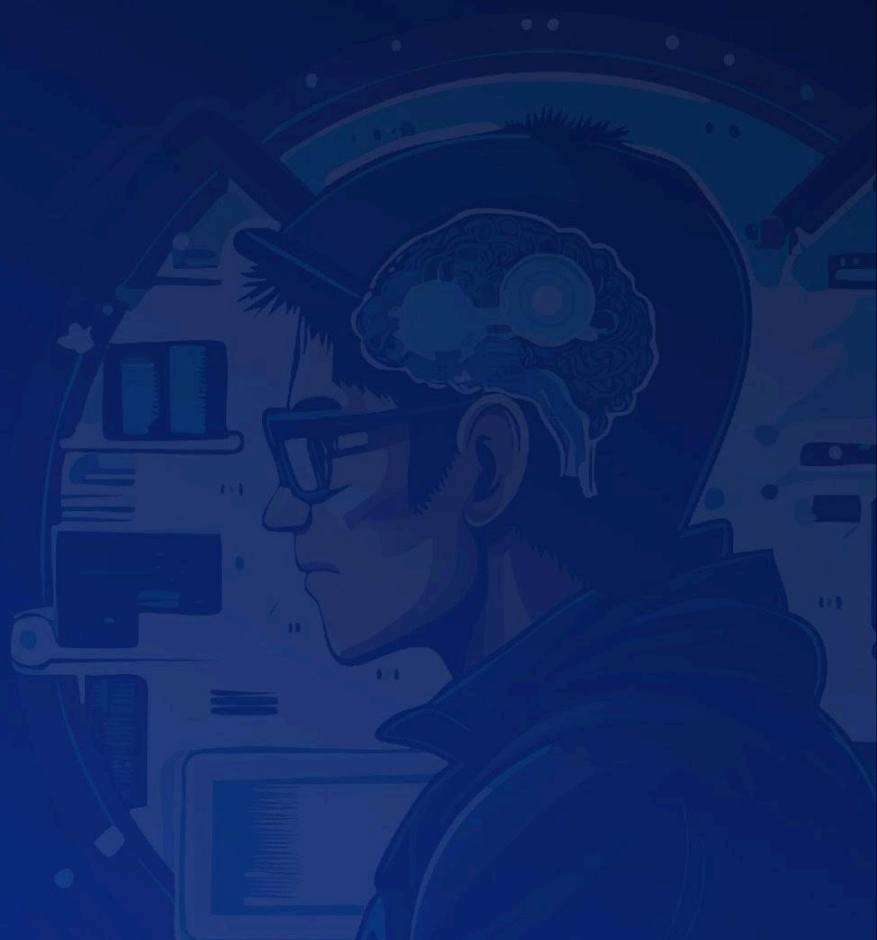
Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	1	0	1
Low	4	3	1
None	1	1	0

Category Breakdown

Suggestion	1
Documentation	0
Bug	5
Optimization	0
Good Code Practices	0

Risk Section Security Review

MoreMarkets Vault



Risk Section

No risks were identified.

Findings

Security Review

MoreMarkets Vault



Findings

3S-Vault-M01

HWM reset during migration causes unfair performance fee on recovery

Id	3S-Vault-M01
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Acknowledged

Description

When a user migrates from the old vault to the new vault via **MoreVaultMigrator.finalizeMigration**, their High Water Mark per Share (HWMpS) in the new vault is initialized to the current price per share at deposit time.

This creates a scenario where users pay performance fees on asset recovery that does not constitute real profit:

- 1) User deposits \$100k in the old vault
- 2) Underlying asset value drops - position is now worth \$80k
- 3) Migration occurs: **oldVault.redeem** returns \$80k, **newVault.deposit()** mints shares at the new vault's current PPS
- 4) In the new vault, **_updateUserHWMpS** sets HWMpS to the current price per share (effectively 1.0 in a fresh vault), reflecting the \$80k entry
- 5) Asset value recovers to \$100k — PPS rises to 1.25 in the new vault
- 6) The vault calculates \$20k in "profit" above HWMpS and charges a performance fee

Had the user remained in the old vault, the same price recovery to \$100k would not represent any realized profit — the user simply returns to their original deposit value. Performance fees should only apply to gains above a user's historical peak, and a return to break-even is not a gain.

The loss to the user is proportional to the drawdown at migration time. At 20% fee rate, a user migrating a \$100k position during a 20% drawdown pays \$4k in fees on zero real profit. At 50% drawdown, the fee is \$10k.

Recommendation

A couple of options:

- 1) Carry over HWMpS during migration. Add a privileged function to the new vault that allows the migrator contract to set a user's HWMpS to their old vault value (adjusted for any PPS ratio difference between vaults). This ensures the user's fee baseline is preserved across migration.
- 2) Acknowledge and document the behavior: If the HWMpS reset is accepted as a design trade-off, document it explicitly so that users can make an informed decision about migration timing. Consider giving users the ability to trigger their own migration (rather than curator-only) so they can choose to migrate after asset recovery, when the HWMpS reset has minimal impact.

Client note

Acknowledged, migration is voluntary. User creates the request, can cancel anytime, and needs to approve allowance. The HWM reset is expected behavior given the redeem/deposit pattern.

The curator also controls when `finalizeMigration` executes, so both sides can ensure migration happens when $PPS \geq$ the user's historical HWM. The user does not lose money since vaults are not paused during migration and the number of shares received reflects the new vault's current PPS at execution time, not a loss of value.

Carrying over HWM across vaults would also create an unfair situation for existing users in the new vault. A migrating user with a high HWM from the old vault would avoid performance fees while other users who deposited directly into the new vault pay normally from their entry point. Additionally as users migrate at different times the vault dynamics change with each migration, making any inherited HWM less meaningful and potentially leaving late migrators facing reduced liquidity and a different PPS context entirely.

Users should be informed to migrate only when $PPS \geq$ their historical HWM. Exposing/transferring HWM across vaults adds too much complexity for this case.

3S-Vault-L01

External trust: curator controls slippage protection

Id	3S-Vault-L01
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Acknowledged

Description

MoreVaultMigrator::finalizeMigration includes a **minNewShares** parameter and reverts when **newSharesMinted < minNewShares**.

Although users may be able to specify slippage when creating the migration flow, the function is **onlyCurator**, and the effective slippage protection therefore depends on the curator providing a correct **minNewShares** value.

This introduces an external trust assumption: if the curator sets **minNewShares** too low, the migration can mint fewer new shares than intended (worse-than-expected exchange rate). The impact is limited because the vault itself has no native slippage protection and the user already has limited choices during this process.

Recommendation

Document clearly that the effective slippage protection in **finalizeMigration** depends on curator-supplied parameters, so users and operators understand the trust assumption.

Client note

Acknowledged, **finalizeMigration** is intentionally **onlyCurator**. The curator is responsible for choosing the right moment to execute since vaults are not paused during migration, positions may be partially liquid, and both vault states can differ at execution time. Centralizing execution in the curator is by design.

There is no real loss for the user. Receiving fewer shares does not mean losing value since shares reflect the current PPS of the new vault at the time of migration. The number of shares varies depending on when the curator executes, not because of dilution.

minNewShares serves as an operational sanity check for the curator, not a user-facing slippage guarantee. Users already trust the curator for execution timing in this flow.

3S-Vault-L02

Document migrator behavior while whitelist mode is enabled

Id	3S-Vault-L02
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in .

Description

VaultFacet::deposit applies whitelist-based deposit limits using the caller address in the normal execution path. **MoreVaultMigrator::finalizeMigration** deposits into the new vault on behalf of the user, but the actual caller is still the migrator contract. If whitelist mode is enabled, this difference can cause migrations to fail unexpectedly or mint fewer shares than users expect because the deposit is evaluated against the migrator's permissions rather than the user's intended allowance.

Recommendation

Document this behavior near the deposit call and clarify that blindly whitelisting the migrator is not a safe workaround, because it could open an unintended path for unwhitelisted deposits through migration flow.

3S-Vault-L03

Missing **CuratorSet** event emission during initialization

Id	3S-Vault-L03
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Addressed in #4549861 .

Description

MoreVaultMigrator::constructor assigns the initial **curator**, but this setup step is not reflected in the event logs because **CuratorSet** is only emitted later from **setCurator**. As a result, integrations or operators that rely on events to reconstruct configuration changes may miss the initial curator value and have an incomplete audit trail for the migrator's deployment state.

Recommendation

Emit **CuratorSet(address(0), _curator)** in the constructor immediately after assigning **curator**.

3S-Vault-L04

redeem return value can be wrong when withdrawal fees are enabled

Id	3S-Vault-L04
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Addressed in #f44d3ed .

Description

VaultFacet::redeem can apply a withdrawal fee before assets are transferred out. In that case, the value returned by **redeem** is not a reliable representation of the amount that **MoreVaultMigrator::finalizeMigration** actually receives. The migrator currently trusts that return value as the amount available for the next deposit step.

If fees are active, this mismatch can cause the migration deposit to revert or consume any leftover token balance already held by the migrator, creating a denial-of-service risk and making migrations behave unpredictably.

Recommendation

In **MoreVaultMigrator::finalizeMigration**, derive **assetsReceived** from the token balance difference before and after **redeem** instead of trusting the function return value.

3S-Vault-N01

Redundant owner zero-address check in initialization

Id	3S-Vault-N01
Classification	None
Category	Suggestion
Status	Addressed in #b85bcda .

Impact: None

Likelihood: High

Description

MoreVaultMigrator::constructor verifies that **_owner** is not the zero address before passing the same value to **Ownable**. This extra validation does not change runtime behavior because **Ownable** already reverts when initialized with a zero owner. Keeping both checks makes the initialization logic slightly harder to read and maintain without adding extra protection.

Recommendation

Remove the redundant zero-address check and rely on **Ownable**'s existing validation.